

Arboles en Monton

HeapSort

Mauro Maldonado



Heaps

Mauro Maldonado

Introducción

La estructura heap es frecuentemente usada para implementar *colas de prioridad*. En este tipo de colas, el elemento a ser eliminado (borrados) es aquél que tiene mayor (o menor) prioridad. En cualquier momento, un elemento con una prioridad arbitraria puede ser insertado en la cola. Una estructura de datos que soporta estas dos operaciones es la *cola de prioridad máxima (mínima)*.

Se asume que un elemento es una estructura con una miembro dato *key* además de otros miembros datos.

Por otra parte se sabe que cualquier estructura de datos que implemente una *cola de prioridad maxima* tiene que implementar las operaciones *insert* y *delete*.(insertar, eliminar)

Definición

Un *max (min) tree* es un árbol en el cual el valor de la llave de cada nodo no es menor (mayor) que la de los valores de las llaves de sus hijos (si tiene alguno). Un *max heap* es un árbol binario completo que es también un *max tree*. Por otra parte, un *min heap* es un árbol binario completo que es también un *min tree*.

De la definición se sabe que la llave de la *root (raíz)* de un *min tree* es la menor llave del árbol, mientras que la del *root* de un *max tree* es la mayor. Las operaciones básicas de un heap son:

- Creación de un heap vacío

- Inserción de un nuevo elemento en la estructura heap.

- Eliminación del elemento más grande del heap.

Categorías de un heap

Existen tres categorías de un heap: *max heap*, *min heap* y *min-max heap*.

Un heap tiene las siguientes tres propiedades:

- ◆ Es completo, esto es, las hojas de un árbol están en a lo máximo dos niveles adyacentes, y las hojas en el último nivel están en la posición *leftmost*.
- ◆ Cada nivel en un heap es llenado en orden de izquierda a derecha.
- ◆ Está parcialmente ordenado, esto es, un valor asignado, llamado *key* del elemento almacenado en cada nodo (llamado *parent*), es menor que (mayor que) o igual a las llaves almacenadas en los hijos de los nodos izquierdo y derecho.

Si la llave (*key*) de cada nodo es mayor que o igual a las llaves de sus hijos, entonces la estructura heap es llamada *max heap*.

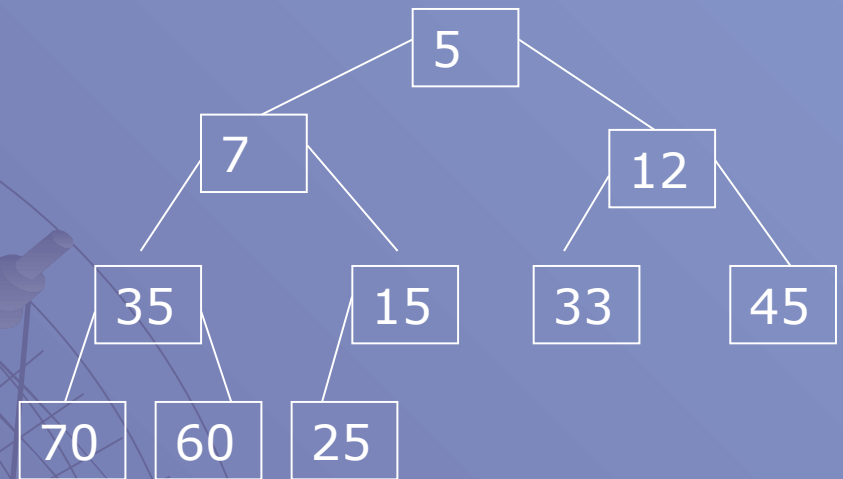
Si la llave (*key*) de cada nodo es menor que o igual a las llaves de sus hijos, entonces la estructura heap es llamada *min heap*.

En una estructura *min-max heap*, un nivel satisface la propiedad *min heap*, y el siguiente nivel inferior satisface la propiedad *max heap*, alternadamente. Un *min-max heap* es útil para colas de prioridad de doble fin.

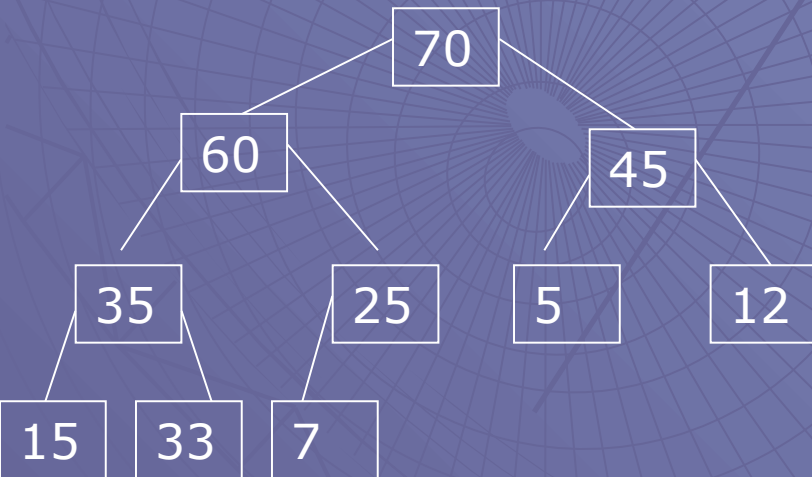
Ejemplo de los tres tipos de heap
para el mismo conjunto de valores
key:

$A = \{33, 60, 5, 15, 25, 12, 45, 70, 35, 7\}$

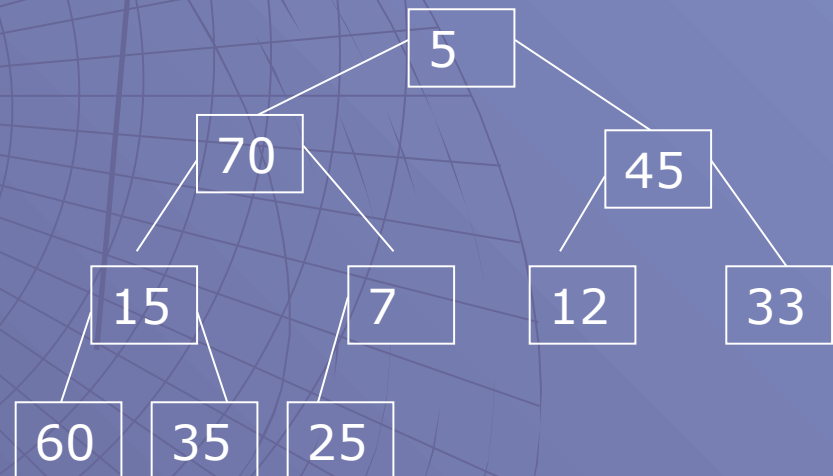
Min heap



Max heap



Min - Max heap



Niveles

Min

Max

Min

Max

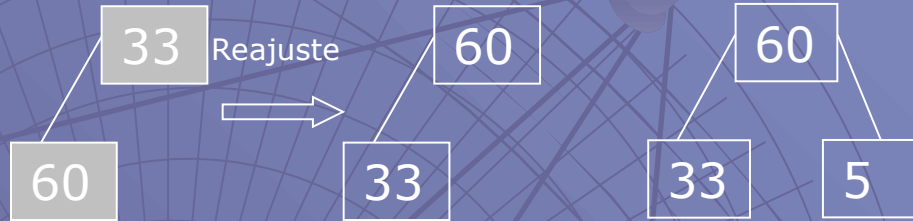
Insert

Dado una lista (arreglo) de n elementos con llaves

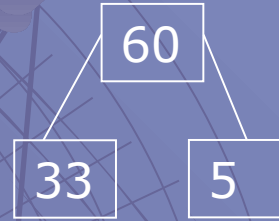
$A = \{33, 60, 5, 15, 25, 12, 45, 70, 35, 7\}$ para ser ordenados en orden ascendente usando el algoritmo heap sort, se construye la estructura *max heap*.



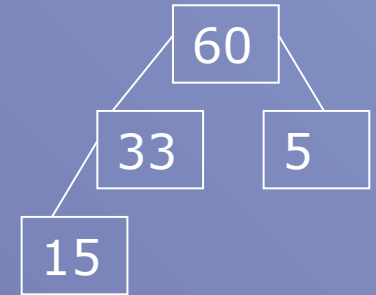
a) Insertar 33 en la estructura vacía



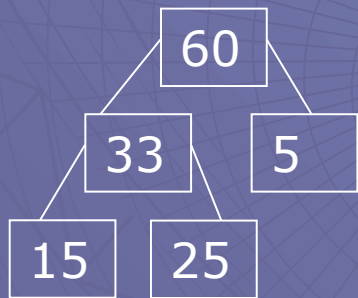
b) Insertar 60



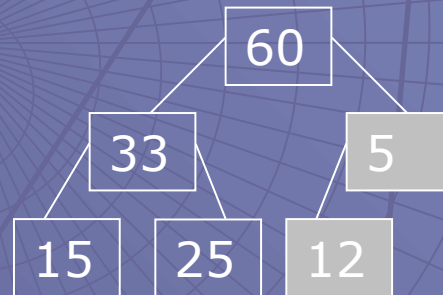
c) Insertar 5



d) Insertar 15

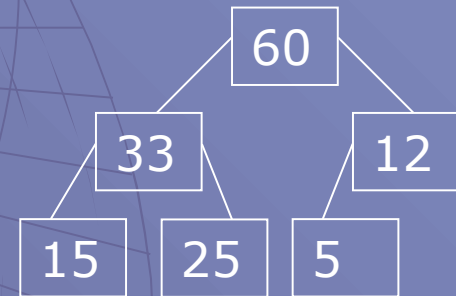


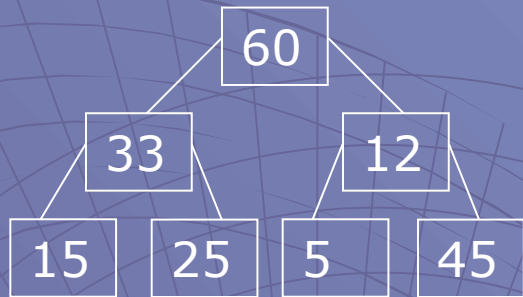
e) Insertar 25



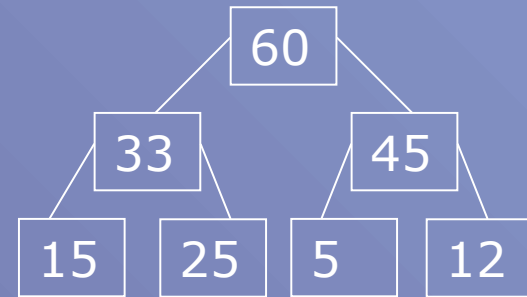
f) Insertar 12

Reajuste

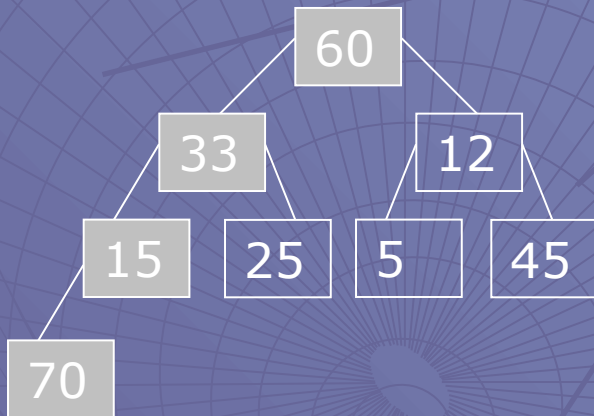




Reajuste

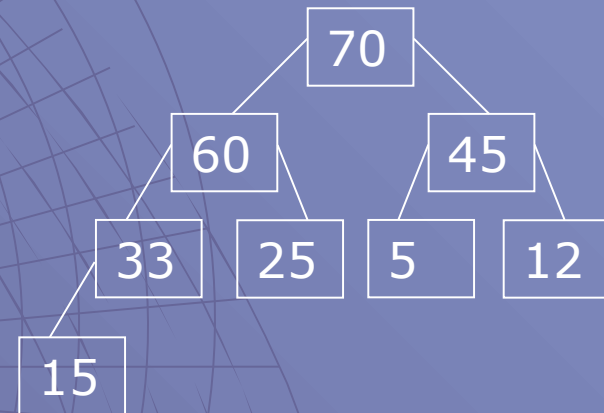


g) Insertar 45

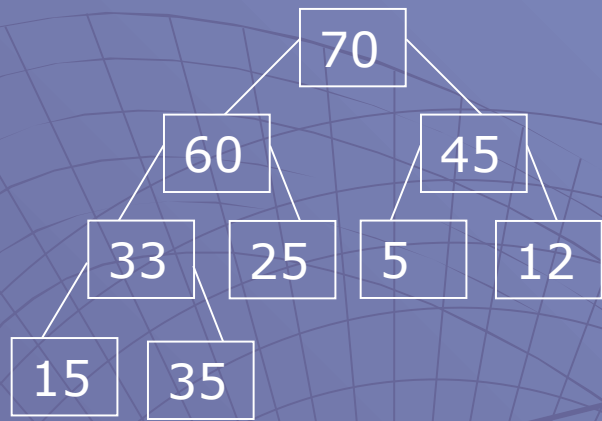


h) Insertar 70

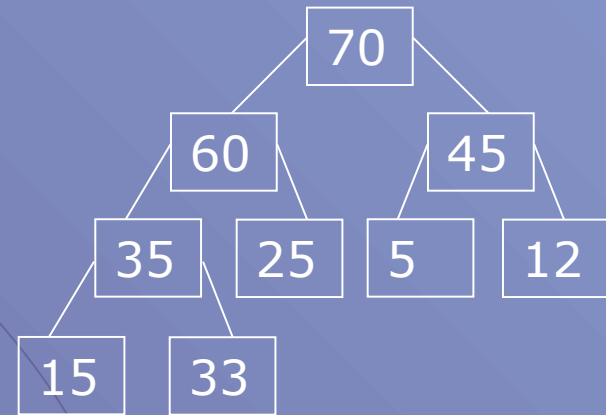
Reajuste



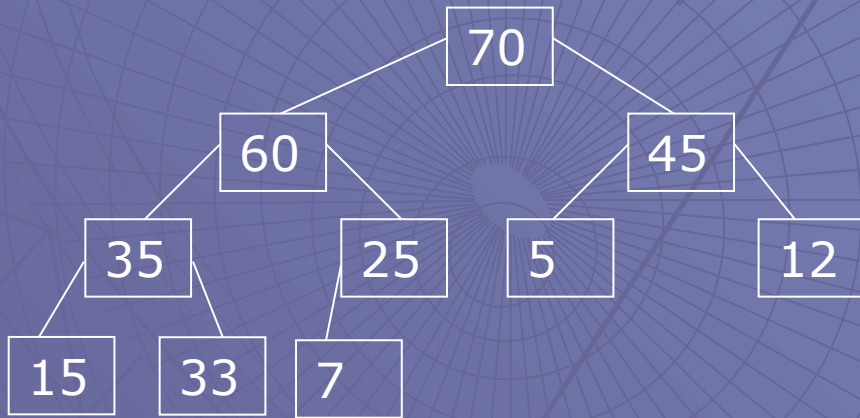
Al final, el elemento con la llave más alta quedará como el *root* de la estructura *max heap*.



Reajuste



i) Inserir 35



j) Insert 7

ListaOrdenada: 70,60,45,35,33,25,15,12,7,5

Para construir el *max heap* se comienza poniendo la primera llave *key* en el nodo *root*, se añade después los siguientes dos nodos como los hijos izquierdo y derecho del nodo *root*, se añade los siguientes cuatro elementos de A como los hijos de segundo nivel (de izquierda a derecha y manteniendo la propiedad *max heap*), se continúa el proceso hasta que se insertan todos los elementos de A.

Todo los nodos en el nivel más bajo son hojas. El algoritmo para construir un *max heap* de un arreglo A de n elementos y ordenarlo con cada hijo menor que o igual a su padre es el siguiente:

Paso 1. Se comienza con A[0] como el nodo root de la estructura heap.

Paso 2. Ahora se comienza a añadir el hijo izquierdo del root.

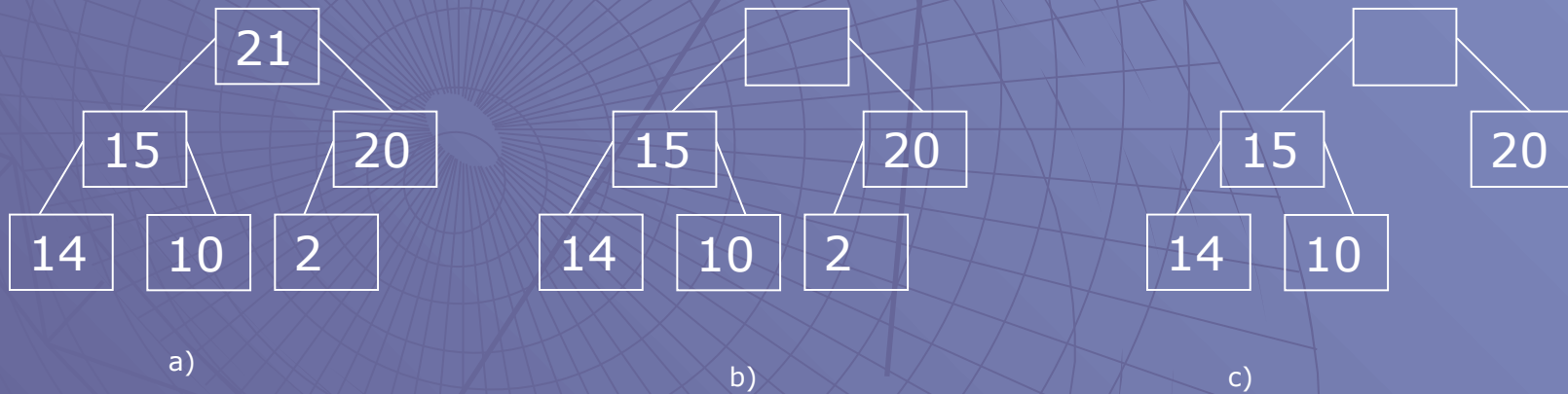
Paso 3. Si el último elemento de A es alcanzado, exit; de otra manera hacer Paso 4.

Paso 4. Comparar cada llave de cada hijo con la de su padre. Si la llave del hijo es mayor, hacer (a) hasta (c): (a) intercambiar (swap) el padre y el hijo, (b) mover hacia arriba el padre y su padre, (c) regresar a paso 3.

Paso 5. Moverse hacia el siguiente hijo y regresar al paso 3.

Eliminar

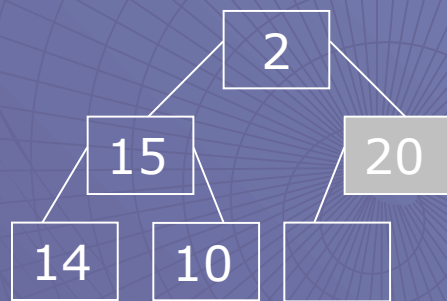
Cuando se necesita eliminar un elemento de la estructura *max heap* se toma de la raíz *root* del heap. Por ejemplo, al eliminar la raíz *root* del siguiente ejemplo (a) resulta que se elimina entonces el elemento 21 (b). Ya que el *heap* resultante tiene solamente cinco elementos, el árbol binario necesita ser reestructurado para que corresponda a un árbol binario completo con cinco elementos. Para hacer esto, se elimina el elemento en la posición 6 (en este caso el elemento 2). Ahora tenemos la estructura correcta (c), pero la raíz *root* está vacante y el elemento 2 no está en el *heap*.



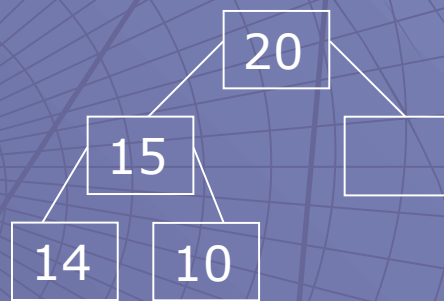
Eliminar elemento 21

Lista Ordenada: 21,

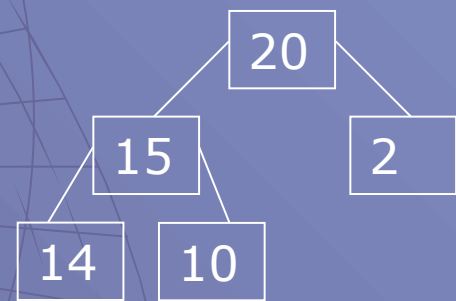
Si se inserta entonces el elemento 2 en la raíz, el árbol binario resultante no es un *max heap*. El elemento en la raíz debe ser el mayor de los dos y los elementos de los hijos izquierdo y derecho de la raíz *root*. Este elemento es el 20. Se mueve entonces a la raíz, y se crea entonces una vacante en la posición 3. Ya que esta posición no tiene hijos, el 2 puede ser insertado aquí (d)



b)

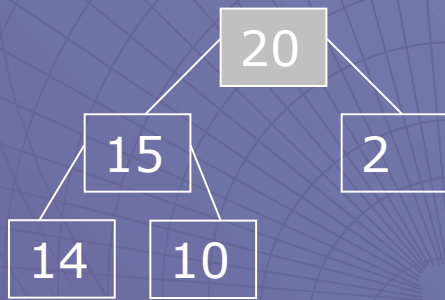


c)

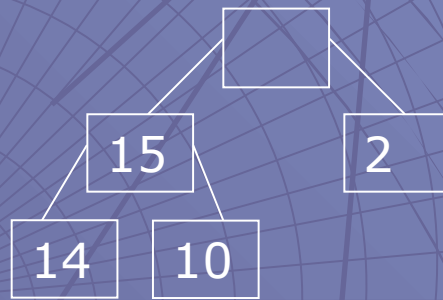


d)

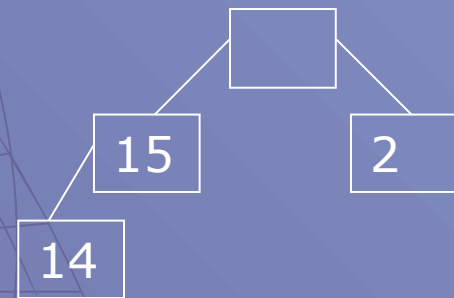
Supongamos ahora que se desea realizar otra eliminación (a). El elemento a borrar ahora es el 20 (b). Siguiendo el borrado, el *heap* tiene ahora la estructura (c). Para llegar a esta estructura el 10 es removido de la posición 5. Este no puede ser insertado en la raíz ya que no es el máximo. El 15 es el máximo y lo movemos a la raíz (d), podríamos poner entonces el 10 en la posición 2; pero no podríamos ya que éste (el elemento 10) es menor que el 14 debajo de éste. Así que movemos hacia arriba el 14 (e) y el 10 es insertado en la posición 4 (f)



a)



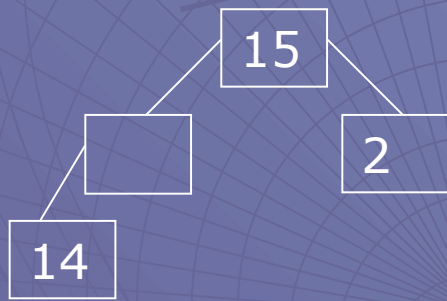
b)



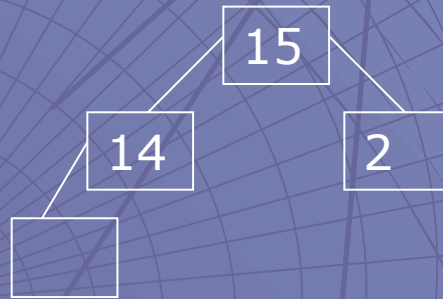
c)

Lista Ordenada: 21,20

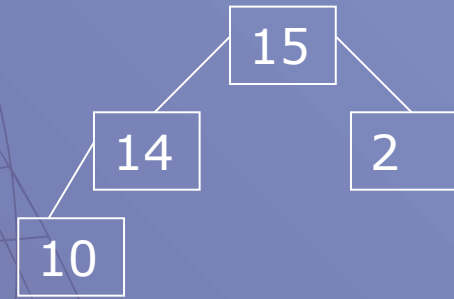
Supongamos ahora que se desea realizar otra eliminación (a). El elemento a borrar ahora es el 20 (b). Siguiendo el borrado, el *heap* tiene ahora la estructura (c). Para llegar a esta estructura el 10 es removido de la posición 5. Este no puede ser insertado en la raíz ya que no es el máximo. El 15 es el máximo y lo movemos a la raíz (d), podríamos poner entonces el 10 en la posición 2; pero no podríamos ya que éste (el elemento 10) es menor que el 14 debajo de éste. Así que movemos hacia arriba el 14 (e) y el 10 es insertado en la posición 4 (f)



d)



e)



f)

Lista Ordenada: 21,20,15

Bibliografía

- E. Horowitz, S. Sahni y D. Mehta. *Fundamentals of Data Structures in C++*. Freeman, EUA, 1995.
- S. Sengupta, C. Korobkin. *C++, Object-Oriented Data Structures*. Springer-Verlag, EUA, 1994.